

Ionirix Image Generation Architecture v1

Unified Illustrious XL Pipeline with Ion Orchestration

Path: /docs/image-gen-architecture-v1.md

Author: Mirnes — Sole Developer / Architect

Date: May 8, 2026

Status: Implementation-Ready — v1.0

Platform: Ionirix Sovereign Coordination Platform

1. Executive Summary

1.1 Problem Statement

The current Ionirix image generation stack is fragmented across two independent model backends—Black Forest Labs Flux 1 Schnell and Stability AI Stable Diffusion—with no unified pipeline, no shared prompt conventions, no consistent quality baseline, and no deep Ion orchestration layer. Ion currently functions as a thin proxy that passes prompts through to whichever model is selected, adding no reasoning, no tag expansion, no style inference, and no parameter optimization. This doubles infrastructure complexity while delivering inconsistent, style-limited output—particularly in anime and illustration domains where neither base model excels.

1.2 Solution

Unify around the **Illustrious XL ecosystem** (SDXL-based, anime-optimized) with **ComfyUI running headless** as the model gateway, and **Ion as a first-class orchestration layer** that performs multi-step reasoning, prompt refinement, style routing, and workflow assembly before every generation call. A single model family.

A single pipeline. A single prompt convention. Ion does the thinking; ComfyUI does the rendering.

1.3 Key Architectural Decisions

Decision	Choice	Rationale
Primary model family	Illustrious XL (SDXL architecture)	State-of-the-art anime quality, massive ecosystem, Danbooru tag support
Default checkpoint	NoobAI-XL v-pred 1.0	Best overall anime checkpoint, deepest character knowledge, v-prediction quality
Model serving layer	ComfyUI headless API	Workflow-based, checkpoint hot-swap, LoRA stacking, mature ecosystem
Orchestration	Ion multi-step reasoning pipeline	NL→tags, style inference, parameter optimization, safety filtering
Job queue	Redis-backed async queue	Priority levels, timeout handling, retry logic
Future migration path	FLUX.2 [klein] 4B (Apache 2.0)	Sub-second generation, but anime ecosystem not yet mature

2. Model Recommendation & Justification

2.1 Primary Engine: Illustrious XL Ecosystem

The **Illustrious XL** model family, developed by OnomaAI Research, is the foundational engine for this architecture. Illustrious is a continued training of the SDXL architecture (3.5B parameter UNet) specifically optimized for illustration and animation generation.

Key technical facts:

- **Architecture:** Stable Diffusion XL (SDXL) base, 3.5B parameter UNet, dual CLIP text encoders (OpenCLIP ViT-bigG + CLIP ViT-L)
- **Training data:** Fine-tuned on the comprehensive Danbooru2023 dataset with refined multi-level captions covering all tags and natural language
- **Anime quality:** Demonstrates state-of-the-art performance in animation-style generation per the technical report (arXiv:2409.19946)
- **High resolution:** Supports generation over 20MP with proper methods (multi-resolution training strategy)
- **Prompt system:** Booru-style tag prompting (Danbooru tags) for precise character, style, and composition control
- **Ecosystem:** Massive library of fine-tunes, LoRAs, and community checkpoints on CivitAI—18,600+ reviews, 43.9M image generations on the base model alone
- **License:** Open-source under Illustrious License (fair public AI license, discourages closed-source monetization)

Why SDXL Architecture Matters

SDXL's dual text encoder architecture provides superior prompt understanding compared to single-encoder models. The OpenCLIP ViT-bigG handles complex compositional prompts while CLIP ViT-L captures fine-grained details. This is critical for Danbooru tag-based workflows where precise tag interpretation drives output quality.

2.2 Default Checkpoint: NoobAI-XL (v-prediction)

The production default checkpoint is **NoobAI-XL v-pred 1.0**, developed by Laxhar Lab. This is the single most recommended anime checkpoint in the Illustrious ecosystem as of May 2026.

Property	Value
Full name	NoobAI-XL (NAI-XL) V-Pred-1.0-Version
Developer	Laxhar Lab
Base model	Illustrious XL (NoobAI base)
Prediction type	V-Prediction (distinct from epsilon-prediction)
File size	6.62 GB (bf16 SafeTensor)
VRAM requirement	8 GB+ (fp16), 13 GB+ recommended for quality
Training data	Full Danbooru + e621 datasets with native tags and NL captions
CivitAI reviews	Overwhelmingly Positive (11,235+)
Recommended samplers	Euler, DDIM (does not support Karras series)
Recommended CFG	5.0 (with Dynamic CFG plugin, rescale 0.2)
Recommended steps	28
Quality tags	masterpiece, best quality, absurdres

Why NoobAI-XL v-pred:

- **V-Prediction technology** delivers superior image quality, detail, and color accuracy compared to epsilon-prediction variants
- **Deepest character knowledge** of any open-source anime checkpoint—trained on full Danbooru + e621
- **Best prompt adherence**—quality tags have pronounced effect, flexible style combination weights

- **Anatomical accuracy**—fine-tuned on high-quality datasets for compositional rationality
- **Full LoRA compatibility** with the entire Illustrious XL ecosystem

V-Prediction Configuration Requirement

NoobAI-XL v-pred operates differently from standard epsilon-prediction models. In ComfyUI, the workflow must explicitly set `prediction_type: "v_prediction"` and `rescale_betas_zero_snr: true`. Use the Dynamic CFG & CFG Rescale plugin with a rescale value of 0.2 to prevent oversaturation or overly gray images. Karras-series schedulers are **not supported**.

2.3 Alternative Checkpoints (Swappable)

These checkpoints are drop-in alternatives within the same Illustrious XL ecosystem. They share LoRAs and use the same ComfyUI pipeline—only prompt conventions and default parameters differ.

Checkpoint	Developer	Best For	Quality Tags	Notes
AnimagineXL 4.0	Cagliostro Research Lab	Clean out-of-box anime	masterpiece, high score, great score, absurdres	Trained on 8.4M anime images, cleanest default output, unique quality tag vocabulary
WAI-Illustrious-SDXL	WAI Community	Character accuracy, vibrant colors	masterpiece, best quality, absurdres	Highest-rated on CivitAI, exceptional at known characters
Pony Diffusion V6 XL	PurpleSmartAI	Cartoon, furry, massive LoRA library	score_9, score_8_up, score_7_up, source_anime	Completely different score-based quality tag system; largest LoRA

Checkpoint	Developer	Best For	Quality Tags	Notes
				ecosystem
Diving-Illustrious Anime	Community	Minimal-prompt generation	masterpiece, best quality, absurdres	Extremely forgiving—hard to get bad outputs even with simple prompts

2.4 Future Path: FLUX.2

Black Forest Labs released **FLUX.2 [klein] 4B** in January 2026—a 4-billion parameter rectified flow transformer under the Apache 2.0 license. Key facts:

- Sub-second generation on consumer GPUs (RTX 3090/4070, ~13 GB VRAM)
- Unified text-to-image and image editing in a single architecture
- Step-distilled and guidance-distilled for real-time workflows
- Fully open weights for commercial use

Why not now: The anime fine-tune ecosystem on FLUX architecture is thin as of May 2026. No equivalent to NoobAI-XL or the Illustrious LoRA library exists for FLUX. Once the community produces anime-quality fine-tunes on FLUX.2, migration is the clear path—the speed advantage is transformative. Monitor quarterly.

2.5 Why Not Stay on Flux 1 Schnell / SD

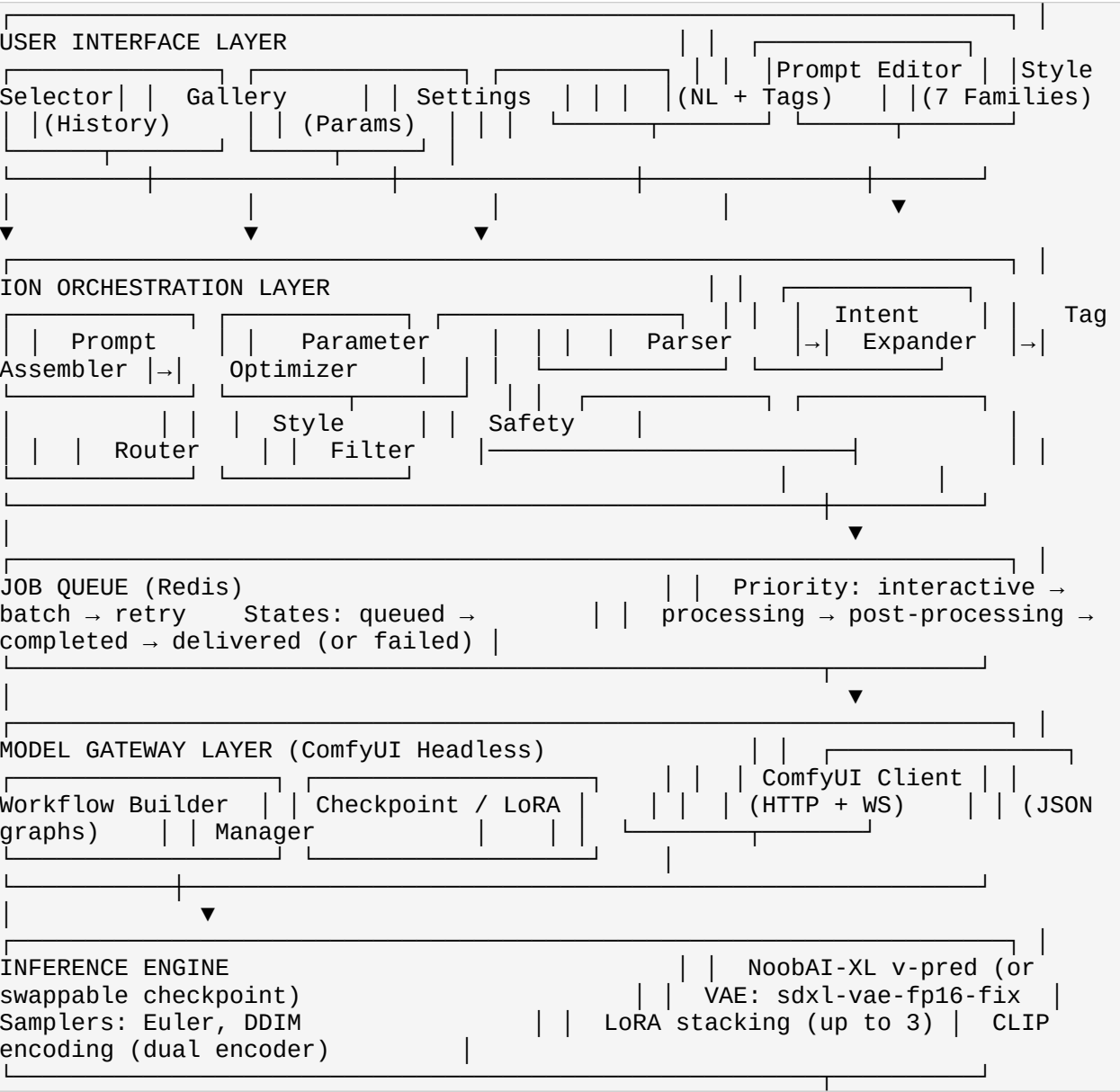
- **Flux 1 Schnell:** Fast (4-step generation) but limited in anime quality. No Danbooru tag training. Thin fine-tune ecosystem. No v-prediction support.
- **Raw Stable Diffusion base:** Generalist model lacking the anime-specific training data, tag conventions, and aesthetic fine-tuning of Illustrious.
- **Dual-pipeline complexity:** Maintaining two model backends with different prompt formats, different parameter spaces, different LoRA ecosystems, and different quality characteristics doubles infrastructure cost with no quality benefit.

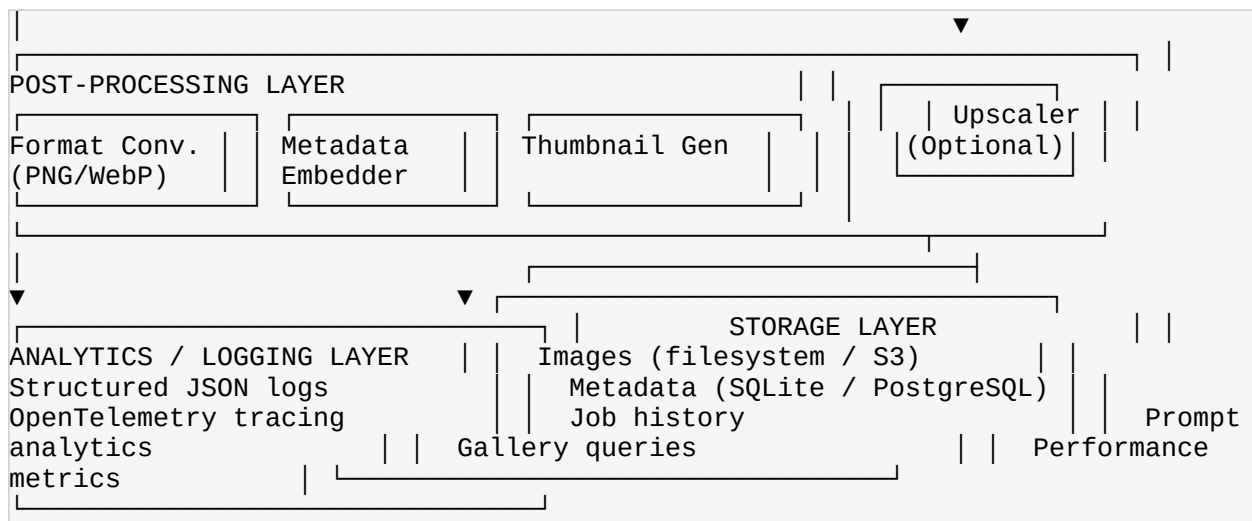
- **Illustrious XL unifies:** Anime excellence + SDXL's mature tooling (ComfyUI, LoRAs, ControlNet, IP-Adapter) + single prompt convention (Danbooru tags) + single pipeline.

3. System Architecture

3.1 Architecture Overview

The system is organized into seven horizontal layers, each with clearly defined responsibilities and interfaces.





3.2 Backend Services & Modules

3.2.1 Model Gateway (ComfyUI Headless)

ComfyUI runs in headless/API mode as the model serving layer. Every generation request is expressed as a ComfyUI workflow graph (JSON), submitted via API, and tracked via WebSocket.

Endpoint	Method	Purpose
<code>/prompt</code>	POST	Submit a workflow graph for execution. Returns <code>prompt_id</code> .
<code>/history/{prompt_id}</code>	GET	Fetch execution history and output file references for a prompt.
<code>/view</code>	GET	Retrieve generated image by filename, subfolder, and type.
<code>/queue</code>	GET	Get current queue state (running + pending prompts).
<code>/interrupt</code>	POST	Interrupt currently running prompt execution.
<code>/ws</code>	WebSocket	Real-time progress updates (step count, preview images, status).

Core responsibilities:

- Workflow-based execution: each generation request is a directed acyclic graph of ComfyUI nodes
- Checkpoint hot-swapping via workflow node configuration (no server restart)
- LoRA stacking support—up to 3 LoRAs per generation, injected as workflow nodes
- Health check: verify ComfyUI is responding, check loaded model, report VRAM usage
- WebSocket streaming for real-time step progress and intermediate latent previews

3.2.2 Ion Orchestration Service

Ion is the reasoning engine. It receives raw user intent and performs a multi-step reasoning pipeline before constructing and submitting the ComfyUI workflow. Ion is **not** a thin proxy—it is the most complex component in the pipeline.

Orchestration pipeline (executed for every generation request):

1. **Intent Parsing** — Extract subject, action, mood, setting, characters from natural language
2. **Profile Check** — Load user's default style preference, favorite checkpoint, safety toggles
3. **Style Inference** — Match to style family based on user selection + mood keywords + profile defaults
4. **Tag Expansion** — Convert natural language concepts to Danbooru tags using Ion's tag knowledge base
5. **Quality Tag Injection** — Prepend checkpoint-appropriate quality tags (NoobAI vs Pony vs AnimateXL conventions)
6. **Negative Prompt Construction** — Merge style preset negatives + safety negatives + user exclusions
7. **Safety Gate** — Scan expanded prompt against content policy

8. **Parameter Optimization** — Select resolution, steps, CFG, sampler based on style family + quality target
9. **Workflow Assembly** — Construct the complete ComfyUI workflow JSON with all nodes wired
10. **Submission** — Enqueue to job queue with priority level

Example reasoning chain: User says *“draw my character in a sunset.”* Ion infers: anime style from user profile → expands to Danbooru tags (1girl, sunset, orange sky, backlighting, silhouette, golden hour) → adds quality tags (masterpiece, best quality, absurdres) → constructs negative prompt → selects 1024×1536 portrait resolution → selects Euler sampler at 28 steps → assembles ComfyUI workflow → submits.

3.2.3 Job Queue

- **Backend:** Redis-backed async queue (Bull or BullMQ for Node.js)
- **Job states:** queued → processing → post-processing → completed → delivered (or failed at any stage)
- **Priority levels:** interactive (user waiting, highest priority), batch (background generation), retry (automatic retry of failed jobs)
- **Timeout handling:** 120s default, 300s for high-res upscale jobs
- **Concurrency:** configurable max concurrent jobs (default 2 for single-GPU setups)
- **Dead letter queue:** failed jobs after max retries are moved to DLQ for manual inspection

3.2.4 Prompt Preprocessor

- **Tag normalization:** Lowercase, underscores, comma-separated Danbooru format. Deduplicate tags.
- **Quality tag injection:** Based on active checkpoint—NoobAI uses masterpiece, best quality, absurdres; AnimateXL uses masterpiece, high score, great score, absurdres; Pony uses score_9, score_8_up, score_7_up

- **Negative prompt assembly:** Style preset base negatives + anatomy safety net (bad hands, extra fingers, bad anatomy) + quality floor (worst quality, low quality) + user overrides
- **Resolution optimization:** Map aspect ratio intent (portrait, landscape, square) to SDXL-optimal bucket resolutions (1024×1536, 1536×1024, 1024×1024, 1024×768, 768×1024)
- **LoRA weight injection:** Validate requested LoRAs exist, check compatibility, inject as workflow nodes with specified weights

3.2.5 Safety & Validation

- **Input validation:** Prompt length limit (500 tokens positive, 300 tokens negative), parameter range checks
- **Banned content detection:** Configurable keyword blocklist for prompt input
- **Output validation:** Optional NSFW classifier on generated images (configurable threshold, default 0.7)
- **Rate limiting:** Per-user-session limits (configurable, default 30 generations/hour)
- **Content policy:** Fully configurable per deployment via `safety.config.ts`

3.2.6 Post-Processing Pipeline

Stage	Input	Output	Notes
AI Upscaling (optional)	Raw image tensor	Upscaled image (2x or 4x)	Models: 4x-UltraSharp (general), RealESRGAN-x4plus-anime (anime-optimized)
Format Conversion	Raw/upscaled image	PNG, WebP, or JPEG	PNG default for lossless; WebP for gallery thumbnails; JPEG with quality 85-95

Stage	Input	Output	Notes
Metadata Embedding	Final image + params	Image with EXIF/PNG chunks	Embeds: prompt, negative, seed, model, steps, CFG, sampler, LoRAs
Thumbnail Generation	Final image	256px-wide WebP thumbnail	For gallery grid view, fast loading
Watermark (optional)	Final image	Watermarked image	Configurable text/image overlay, disabled by default

3.2.7 Logging & Metrics

- **Structured logging:** JSON-formatted log entries for every generation request, including full prompt, parameters, timing, and outcome
- **Performance metrics:** Generation time (p50, p95, p99), queue depth, success/failure rate, model load time, VRAM utilization
- **Prompt analytics:** Most-used tags, style distribution, error patterns, retry rates
- **Tracing:** OpenTelemetry-compatible spans across the full pipeline (Ion reasoning → queue → inference → post-processing → delivery)

3.3 Frontend & UI Flows

3.3.1 Prompt Editor

- Rich text input with Danbooru tag autocomplete (fuzzy matching against tag database)
- Tag weight syntax support: (tag:1.2) for emphasis, (tag:0.5) for de-emphasis
- **Natural Language mode:** User writes in plain English; Ion translates to tags behind the scenes
- **Expert mode:** User writes raw Danbooru tags directly; Ion adds quality tags and negatives only

- Prompt history (last 50), favorites/bookmarks, and prompt templates

3.3.2 Style & Preset Selector

- Visual preset cards showing example output thumbnails for each of the 7 style families
- Style families: Cinematic Niji-Grade Anime, Soft Pastel Shoujo, Gritty Seinen, Retro 90s Cel-Anime, Semi-Realistic 2.5D, Painterly/Watercolor, LoFi Aesthetic
- Each preset encodes: positive prompt prefix, negative prompt additions, sampler, CFG scale, steps, default resolution, optional checkpoint override, optional LoRA stack
- One-click application—selecting a style automatically configures all parameters

3.3.3 Model/Checkpoint Selector

- Dropdown showing available checkpoints with thumbnail previews and metadata
- Shows: checkpoint name, base model family, recommended CFG, recommended sampler, file size
- LoRA browser with search by name, category (character/style/concept), and compatibility filter

3.3.4 Generation Progress & Status

- Real-time progress bar (ComfyUI reports step progress via WebSocket: step 14/28)
- Optional live preview showing intermediate denoising steps (TAESD fast decode)
- Queue position indicator with estimated wait time
- Error display with actionable messages (see Error Codes in Section 4.3)

3.3.5 Gallery & History

- Grid view of generated images with metadata overlay on hover (prompt, seed, model, style)
- Filter by: style family, date range, checkpoint, prompt keywords, favorite status
- Favorite/bookmark system for curating best outputs
- **Re-generate:** Exact reproduction with same seed + all settings
- **Variation:** Same prompt, random new seed
- **Edit & Retry:** Open prompt in editor pre-filled with all previous parameters

3.3.6 Error Surfaces

- **Toast notifications:** Transient errors (timeout, queue full)—auto-dismiss after 8 seconds
- **Modal dialogs:** Critical errors (model failed to load, VRAM OOM)—require user action
- **Inline validation:** Prompt editor shows warnings for empty prompt, excessive length, contradictory tags

4. Ion-Engine Contract

4.1 Generation Request Schema

GenerationRequest — Ion → Model Gateway

```
{  "request_id": "uuid-v4",  "user_id": "string",  "session_id": "string",  "priority": "interactive | batch | retry",  "timestamp": "ISO-8601",  "prompt": {    "positive": "string (Danbooru tags + optional NL fragments)",    "negative": "string",    "quality_tags": ["masterpiece", "best quality", "absurdres"],    "style_tags": ["anime", "detailed", "cinematic lighting"]  },  "model": {    "checkpoint": "noobai-xl-vpred-v1.0",    "prediction_type": "v_prediction",    "vae": "sdxl-vae-fp16-fix",    "loras": [      { "name": "style-cinematic", "weight": 0.8 },      { "name": "character-custom", "weight": 0.7 }    ],    "clip_skip": 2  },  "parameters": {    "width": 1024,    "height": 1536,    "steps": 28,    "cfg_scale": 5.0,    "cfg_rescale": 0.2,    "sampler": "euler",    "scheduler": "normal",    "seed": -1,    "batch_size": 1  },  "post_processing": {    "upscale": {      "enabled": false,      "model":
```

```
"4x-UltraSharp",      "scale": 2.0    },      "format": "png",
"quality": 95,         "embed_metadata": true,    "generate_thumbnail":
true  },      "ion_metadata": {      "reasoning_chain": [      "intent_parse",
"profile_check",      "style_infer",      "tag_expand",
"quality_inject",      "negative_assemble",      "safety_gate",
"param_optimize",      "workflow_build"    ],      "original_user_prompt":
"string (raw user input)",      "style_family": "cinematic_niji",
"inferred_mood": "dramatic",      "confidence": 0.92  } }
```

4.2 Generation Response Schema

GenerationResponse — [Model Gateway](#) → [Ion](#) → [User](#)

```
{  "request_id": "uuid-v4",  "status": "completed | failed | partial",
"images": [    {      "url": "/storage/images/{id}.png",
"thumbnail_url": "/storage/thumbs/{id}.webp",      "width": 1024,
"height": 1536,      "format": "png",      "size_bytes": 2048000,
"seed": 12345678    }  ],  "timing": {    "queue_ms": 150,
"model_load_ms": 0,    "inference_ms": 8500,    "post_processing_ms": 1200,
"total_ms": 9850  },  "model_info": {    "checkpoint": "noobai-xl-vpred-
v1.0",    "prediction_type": "v_prediction",    "loras_applied": ["style-
cinematic:0.8", "character-custom:0.7"],    "actual_steps": 28,
"actual_cfg": 5.0,    "vae": "sdxl-vae-fp16-fix"  },  "error": null }
```

Error shape (when status is "failed"):

```
{  "error": {    "code": "E_VRAM_OOM",    "message": "Out of GPU memory at
1536x2048. Try 1024x1536.",    "retriable": true,    "suggested_action":
"reduce_resolution"  } }
```

4.3 Error Codes

Code	Description	Retriable	Suggested Action
E_QUEUE_FULL	Job queue at capacity	Yes	Wait and retry in 10s
E_MODEL_LOAD	Checkpoint failed to load (corrupt, missing, incompatible)	No*	Check file integrity, switch checkpoint
E_VRAM_OOM	Out of GPU memory during inference	Yes	Reduce resolution or batch size
E_TIMEOUT	Generation exceeded timeout (120s / 300s)	Yes	Reduce steps or resolution, retry
E_SAFETY_BLOCK	Content policy violation detected	No	Modify prompt to comply with policy

Code	Description	Retriable	Suggested Action
E_INVALID_PARAMS	Invalid generation parameters	No	Fix parameter values (see validation errors)
E_LORA_NOT_FOUND	Requested LoRA file not available on server	No*	Install LoRA or remove from request
E_POST_PROCESS	Post-processing pipeline failed	Yes	Retry; raw image may still be available
E_COMFYUI_DOWN	ComfyUI server not responding	Yes	Wait for health check recovery

** Retriable after configuration change, not with the same request.*

4.4 Performance Contracts

Scenario	Target Latency	Assumptions
1024×1024 @ 28 steps	< 10s	RTX 4090, model hot in VRAM, fp16
1024×1536 @ 28 steps	< 15s	RTX 4090, model hot in VRAM, fp16
1024×1536 @ 28 steps + 2x upscale	< 20s	Upscale adds 2-5s depending on model
Checkpoint swap (cold → hot)	5-15s	6.5 GB checkpoint load from SSD
Queue drain (normal load)	< 30s	Max 2 concurrent jobs, <10 queued
End-to-end (user click → image displayed)	< 12s typical	Model hot, no queue, no upscale

5. Data Flow & Pipeline Order of Operations

5.1 Full Request Lifecycle

Step 1: User Prompt Submission

User interacts with the frontend and submits a generation request.

Data emitted:

```
{  raw_prompt: "a warrior girl standing on a cliff at sunset",
  selected_style: "cinematic_niji",           // or null for auto-infer
  selected_checkpoint: "noobai-xl-vpred",     // or null for default
  parameter_overrides: { steps: 30 },        // optional user tweaks
  lora_selections: [],                       // optional    user_id: "usr_abc123",
  session_id: "sess_xyz789" }
```

Step 2: Ion Reasoning Pipeline

Ion orchestration node receives the raw user input and executes its 12-step reasoning chain (detailed in Section 6.4). Each step transforms and enriches the request.

Data emitted: Complete `GenerationRequest` (schema from Section 4.1), including fully expanded Danbooru tags, assembled negative prompt, optimized parameters, and constructed ComfyUI workflow JSON.

Step 3: Job Queue Enqueue

Queue node receives the `GenerationRequest`, assigns a queue position based on priority level.

Data emitted to frontend (via WebSocket):

```
{  request_id: "req_uuid",  queue_position: 0,  estimated_wait_ms: 200,
  status: "queued" }
```

Step 4: Model Inference (ComfyUI)

ComfyUI receives the workflow JSON and executes the node graph:

11. Load checkpoint into VRAM (skip if already hot)
12. Load LoRAs and apply weight modifications to UNet + CLIP

13. Encode positive prompt via dual CLIP text encoders
14. Encode negative prompt via dual CLIP text encoders
15. Initialize random latent noise (from seed)
16. Denoise latent via sampler (Euler/DDIM, N steps, CFG guided)
17. Decode final latent → pixel space via VAE

Data emitted:

```
{  raw_image: Buffer (PNG bytes),  seed_used: 12345678,  actual_steps: 28,  inference_time_ms: 8500,  comfyui_prompt_id: "comfy_uuid" }
```

Step 5: Post-Processing

Post-processing node receives the raw image and applies the pipeline:

18. AI upscale (if enabled) → 4x-UltraSharp or RealESRGAN-anime
19. Format conversion → PNG/WebP/JPEG with quality setting
20. Metadata embedding → prompt, seed, model info into EXIF/PNG chunks
21. Thumbnail generation → 256px-wide WebP for gallery
22. Write to storage → filesystem or object store

Data emitted:

```
{  image_url: "/storage/images/req_uuid_0.png",  thumbnail_url:  "/storage/thumbs/req_uuid_0.webp",  storage_path:  "./storage/images/req_uuid_0.png",  file_size_bytes: 2048000,  metadata:  { ... }  // full generation params }
```

Step 6: Delivery

23. Update job status to `completed` in Redis
24. Push `GenerationResponse` to user via WebSocket
25. Insert record into metadata database (gallery/history)
26. Update job status to `delivered`

Step 7: Logging & Analytics

Analytics node receives full pipeline telemetry and logs a structured record:

```
{  request_id, user_id, session_id,  prompt_data: { positive, negative,  quality_tags, style_tags },  model_data: { checkpoint, loras,
```

```
prediction_type },    timing_data: { queue_ms, inference_ms, post_ms, total_ms
},    output_data: { width, height, format, size_bytes, seed },    error_data:
null | { code, message, retrieable },    style_family: "cinematic_niji",
ion_confidence: 0.92 }
```

6. Ion Reasoning & Prompt Adaptation

6.1 How Ion Adapts Prompts

Ion's prompt adaptation is the core differentiator of this architecture. It transforms vague human intent into precise, checkpoint-optimized Danbooru tag sequences.

- **Natural language → Danbooru tag translation:** Ion uses its deep knowledge of anime conventions, character archetypes, and visual concepts to map NL phrases to precise tags. *“warrior girl”* → 1girl, armor, warrior, sword, determined expression
- **Style-aware tag ordering:** Tags are ordered by importance following Illustrious XL's training order: quality → character → appearance → pose → expression → clothing → background → lighting → style
- **Weight balancing:** Ion adjusts tag weights (tag:weight) based on composition complexity. Simple compositions get fewer weighted tags; complex multi-character scenes get explicit weights on key elements.
- **Multi-shot refinement:** If a first generation receives negative quality feedback, Ion can modify the prompt (adjust weights, add/remove tags, change parameters) and retry automatically.

6.2 Prompt Templates by Style Family

Style Family	Positive Prefix	Negative Additions	Sampler	Steps	CFG	Resolution
Cinemat	masterpie	flat	Euler	28	5.0	1024×15

Style Family	Positive Prefix	Negative Additions	Sampler	Steps	CFG	Resolution
ic Niji-Grade Anime	ce, best quality, absurdres, anime, detailed, cinematic lighting, dramatic angle, vivid colors, sharp focus, depth of field, bloom	colors, simple background (when bg detail wanted)				36
Soft Pastel Shoujo	masterpiece, best quality, absurdres, pastel colors, soft lighting, dreamy, shoujo, delicate linework, sparkles, flower motifs, ethereal	harsh shadows, dark colors, gritty, high contrast	Euler	25	5.5	1024×1024
Gritty Seinen	masterpiece, best quality, absurdres, dark atmosphere, mature, gritty, high contrast, seinen, dramatic shadows, muted colors	bright colors, cute, chibi, pastel, sparkles	Euler	30	6.0	1024×1536
Retro 90s Cel-Anime	masterpiece, best quality,	modern anime, flat	Euler	25	5.5	1024×768

Style Family	Positive Prefix	Negative Additions	Sampler	Steps	CFG	Resolution
	absurdres , 1990s anime, cel shading, retro anime, VHS aesthetic , warm tones, nostalgic , film grain	digital colors, overly clean, 3D render				
Semi-Realistic 2.5D	masterpiece, best quality, absurdres , semi-realistic , detailed skin, realistic lighting, anime eyes, detailed hair, subsurface scattering	flat, cartoon, chibi, sketch, 2d	DDIM	30	5.0	1024×1536
Painterly / Watercolor	masterpiece, best quality, absurdres , watercolor, painterly , soft edges, visible brushstrokes, traditional media, artistic	sharp lines, digital art, photorealistic, clean edges	Euler	25	5.5	1024×1024
LoFi Aesthetic	masterpiece, best	harsh lighting,	Euler	25	5.0	1536×1024

Style Family	Positive Prefix	Negative Additions	Sampler	Steps	CFG	Resolution
c	quality, absurdres, lo-fi, cozy, warm lighting, evening, study vibes, headphones, slice of life, detailed background	cold colors, action, fighting				

Base negative prompt (applied to ALL styles):

lowres, bad anatomy, bad hands, text, error, missing fingers, extra digit, fewer digits, cropped, worst quality, low quality, jpeg artifacts, signature, watermark, username, blurry, deformed, disfigured, mutation, mutated, extra limbs, extra arms, extra legs, malformed limbs, fused fingers, too many fingers, long neck

6.3 Checkpoint-Specific Prompt Conventions

Ion must adapt its prompt template when the user selects a different checkpoint. This adaptation is handled by a lookup table in the prompt assembler, keyed by checkpoint family.

Checkpoint Family	Quality Tags	Additional Tags	Notes
NoobAI-XL (v-pred)	masterpiece, best quality, absurdres	None required	Standard Danbooru convention. Set prediction_type: v_prediction, add CFG rescale 0.2.
AnimagineXL 4.0	masterpiece, high score, great score, absurdres	None required	Unique quality vocabulary. Uses high score and great score instead of best

Checkpoint Family	Quality Tags	Additional Tags	Notes
			quality.
Pony Diffusion V6 XL	score_9, score_8_up, score_7_up	source_anime (or source_cartoon, source_furry)	Completely different score-based system. Source tag is mandatory to control output domain.
WAI-Illustrious / Diving-Illustrious	masterpiece, best quality, absurdres	None required	Standard Illustrious convention. Fully compatible with NoobAI tag patterns.

Ion Adaptation Rule

The user never needs to know about prompt convention differences between checkpoints. When switching from NoobAI-XL to Pony V6, Ion automatically replaces masterpiece, best quality with score_9, score_8_up, score_7_up and adds the appropriate source_* tag. This happens transparently in the prompt assembler.

6.4 Ion Chained Reasoning Steps

The following 12-step reasoning chain is executed by Ion before every model call. Each step has a defined input, output, and failure mode.

#	Step	Input	Output	Can Fail?
1	Intent Parse	Raw user prompt (NL)	Structured intent: subject, action, mood, setting, character refs	No (graceful degradation to raw passthrough)

#	Step	Input	Output	Can Fail?
2	Profile Check	user_id	User prefs: default style, checkpoint, NSFW toggle, favorites	No (falls back to system defaults)
3	Style Inference	User selection + intent mood	Style family ID (e.g., cinematic_niji)	No (defaults to Cinematic Niji)
4	Tag Expansion	NL concepts from intent	Danbooru tag list, ordered by importance	No (uses NL as-is if expansion fails)
5	Character Resolution	Character references	Correct character tags + appearance tags + LoRA if available	No (skips if no character detected)
6	Composition Planning	Subject + setting	Framing tags: close-up, full body, wide shot, portrait	No (defaults to upper body)
7	Quality Injection	Checkpoint ID	Quality tag prefix (checkpoint- specific)	No
8	Negative Assembly	Style preset + safety + user exclusions	Complete negative prompt string	No
9	Parameter Optimization	Style family + quality target	Resolution, steps, CFG, sampler, scheduler	No
10	Safety Gate	Expanded positive + negative prompts	Pass / Block (with E_SAFETY_BLOCK)	Yes — blocks request
11	Workflow Build	All assembled data	ComfyUI workflow JSON (node graph)	No
12	Submit	Workflow + metadata	Job queue entry with request_id	Yes — E_QUEUE_FULL

7. VS Code Implementation Plan

7.1 Folder/Module Layout

```
/ionirix |— /src |— /image-gen # Image
generation subsystem root |— /backend |— /gateway
# ComfyUI integration |— comfyui-client.ts # HTTP +
WebSocket client |— workflow-builder.ts # Construct
workflow JSONs |— checkpoint-manager.ts # Checkpoint
loading & hot-swap |— lora-manager.ts # LoRA
discovery & validation |— health.ts # Health
check & model status |— /orchestration # Ion
reasoning pipeline |— ion-image-orchestrator.ts # Main
orchestrator entry point |— intent-parser.ts # NL →
structured intent |— style-router.ts # Style
family routing |— tag-expander.ts # NL concepts →
Danbooru tags |— prompt-assembler.ts # Final prompt
construction |— parameter-optimizer.ts #
Resolution/steps/CFG tuning |— safety-filter.ts #
Content policy enforcement |— /queue # Job
queue management |— job-queue.ts # Redis-backed
async queue |— job-types.ts # Job state machine
& types |— priority.ts # Priority levels &
scheduling |— /post-processing # Output processing
pipeline |— upscaler.ts # AI upscaling
integration |— format-converter.ts # PNG/WebP/JPEG
conversion |— metadata-embedder.ts # EXIF/PNG chunk
metadata |— thumbnail-generator.ts # Thumbnail creation |
|— /storage # Image & metadata persistence |
|— image-store.ts # Filesystem / object storage |
|— metadata-store.ts # SQLite/PostgreSQL metadata |
|— gallery-queries.ts # Gallery/history query layer |
|— /logging # Telemetry & analytics |
generation-logger.ts # Structured generation logs |
metrics-collector.ts # Performance metrics |
analytics.ts # Usage analytics & patterns |— /frontend
|— /components |— PromptEditor.tsx #
Rich prompt input + autocomplete |— StyleSelector.tsx
# Visual style preset cards |— CheckpointSelector.tsx #
Model/checkpoint dropdown |— LoRABrowser.tsx #
LoRA search & selection |— ParameterPanel.tsx #
Advanced parameter controls |— GenerationProgress.tsx #
Progress bar + live preview |— ImageGallery.tsx #
Generated image grid |— ImageDetailModal.tsx # Full-
size view + metadata |— ErrorDisplay.tsx # Error
toast & modal |— /hooks |— useGeneration.ts #
# Generation request & status |— useWebSocket.ts #
WebSocket for live progress |— useGallery.ts #
Gallery data & filtering |— usePresets.ts #
Style preset management |— /pages |
GeneratePage.tsx # Main generation UI |
GalleryPage.tsx # History & gallery view |— /shared
# Shared contracts & types |— types.ts #
```

```

All TypeScript interfaces | | | | schemas.ts # Zod
validation schemas | | | | error-codes.ts # Error code
enum & descriptions | | | | style-presets.ts # Style
family preset definitions | | | | constants.ts #
Shared constants | | | | /config # Configuration
| | | | models.config.ts # Checkpoint & LoRA registry |
| | | | comfyui.config.ts # ComfyUI connection settings |
| | | | queue.config.ts # Redis & queue settings |
safety.config.ts # Content policy rules | | | | env.ts
# Environment variable loader | | | | /comfyui #
ComfyUI installation (gitignored) | | | | /models/checkpoints/ #
Checkpoint .safetensors files | | | | /models/loras/ #
LoRA files | | | | /models/vae/ # VAE files |
/models/upscale_models/ # Upscaler models | | | | /storage
# Generated output (gitignored) | | | | /images/ #
Full-size generated images | | | | /thumbs/ #
Gallery thumbnails | | | | metadata.db # SQLite
metadata database | | | | /docs | image-gen-architecture-v1.md # This
document

```

7.2 Build Order (Implementation Sequence)

Phase 1: Foundation (Week 1-2)

#	File	Description	Dependencies
1	/shared/types.ts	All TypeScript interfaces: GenerationRequest, GenerationResponse, StylePreset, CheckpointConfig, JobState, etc.	None
2	/shared/schemas.ts	Zod validation schemas for all contracts (request, response, config)	types.ts
3	/shared/error-codes.ts	Error code enum with descriptions, retriability, suggested actions	None
4	/shared/style-presets.ts	All 7 style family presets with full parameter sets	types.ts
5	/shared/constants.ts	Resolution buckets, tag limits, timeout values, VRAM	None

#	File	Description	Dependencies
		thresholds	
6	/config/*	All configuration files with sensible defaults	types.ts

Phase 2: Model Gateway (Week 2-3)

#	File	Description	Dependencies
7	comfyui-client.ts	HTTP client (/prompt, /history, /view, /queue) + WebSocket client (/ws for progress)	types.ts, comfyui.config.ts
8	workflow-builder.ts	Constructs ComfyUI workflow JSON from GenerationRequest . Builds node graph: KSampler → VAE Decode → Save Image, with LoRA Loader nodes injected.	types.ts, comfyui-client.ts
9	checkpoint-manager.ts	Lists checkpoints from /comfyui/models/checkpoints/, reports loaded model, triggers hot-swap	comfyui-client.ts, models.config.ts
10	lora-manager.ts	Scans /comfyui/models/loras/, validates compatibility, provides search/filter API	models.config.ts
11	health.ts	Health check: ComfyUI alive, loaded model, VRAM usage, queue depth	comfyui-client.ts

Stubs for GPU-less Development

Create a `MockComfyUIClient` class that implements `IModelGateway` and returns canned responses (a pre-generated test image, fixed timing data, predictable seed). This enables full pipeline development and testing on a CPU-only machine. Swap via environment variable: `COMFYUI MOCK=true`.

Phase 3: Ion Orchestration (Week 3-4)

#	File	Description	Dependencies
12	<code>intent-parser.ts</code>	Parse user NL into structured intent (subject, action, mood, setting, characters)	<code>types.ts</code>
13	<code>tag-expander.ts</code>	Map NL concepts to Danbooru tags. Start with lookup table; later integrate Ion's full reasoning.	<code>types.ts</code>
14	<code>style-router.ts</code>	Select style family from user choice + mood keywords + profile defaults	<code>style-presets.ts</code>
15	<code>prompt-assembler.ts</code>	Merge quality tags + expanded tags + style prefix + negative prompt. Handle checkpoint-specific conventions.	<code>style-presets.ts</code> , <code>models.config.ts</code>
16	<code>parameter-optimizer.ts</code>	Select resolution, steps, CFG, sampler from style preset + user overrides	<code>style-presets.ts</code> , <code>constants.ts</code>
17	<code>safety-filter.ts</code>	Scan prompt against content	<code>safety.config.ts</code>

#	File	Description	Dependencies
		policy, emit pass/block	
18	ion-image-orchestrator.ts	Wire all steps together: 12-step chain from raw input to GenerationRequest	All orchestration modules

Phase 4: Queue & Storage (Week 4-5)

#	File	Description	Dependencies
19	job-types.ts	Job state machine enum, state transition validation	types.ts
20	priority.ts	Priority level enum, scheduling logic	job-types.ts
21	job-queue.ts	Redis-backed queue (BullMQ): enqueue, dequeue, status, cancel, retry	job-types.ts, priority.ts, queue.config.ts
22	image-store.ts	Save/load/list/delete images on filesystem or S3-compatible store	types.ts
23	metadata-store.ts	SQLite (dev) / PostgreSQL (prod) for generation metadata	types.ts
24	gallery-queries.ts	Query layer: filter by style, date, checkpoint, prompt keywords, favorites	metadata-store.ts

Phase 5: Post-Processing & Logging (Week 5-6)

#	File	Description	Dependencies
25	format-converter.ts	PNG/WebP/JPEG conversion using sharp library	None
26	metadata-embedder.ts	Embed generation params into EXIF (JPEG) or PNG text chunks	types.ts
27	thumbnail-generator.ts	Generate 256px-wide WebP thumbnails via sharp	format-converter.ts
28	upscaler.ts	AI upscaling via ComfyUI upscale nodes (RealESRGAN, UltraSharp)	comfyui-client.ts, workflow-builder.ts
29	generation-logger.ts	Structured JSON logging for every request	types.ts
30	metrics-collector.ts	p50/p95/p99 latency, success rate, queue depth metrics	types.ts
31	analytics.ts	Tag frequency, style distribution, error pattern analysis	metadata-store.ts

Phase 6: Frontend (Week 6-8)

#	File	Description	Dependencies
32	useWebSocket.ts	WebSocket hook for ComfyUI progress events	comfyui.config.ts
33	useGeneration.ts	Generation request submission & status tracking hook	types.ts, schemas.ts
34	PromptEditor.tsx	Rich input with tag	useGeneration.ts

#	File	Description	Dependencies
		autocomplete, NL/Expert mode toggle	
35	StyleSelector.tsx	Visual cards for 7 style families with example thumbnails	style-presets.ts, usePresets.ts
36	ParameterPanel.tsx	Steps, CFG, sampler, resolution, seed controls	types.ts
37	GenerationProgress.tsx	Progress bar, step counter, live preview, queue position	useWebSocket.ts
38	ImageGallery.tsx	Grid view with hover metadata, filters, favorites	useGallery.ts
39	ImageDetailModal.tsx	Full-size image, all metadata, re- generate / variation buttons	types.ts
40	GeneratePage.tsx	Main page: PromptEditor + StyleSelector + ParameterPanel + Progress	All components
41	GalleryPage.tsx	Gallery page: ImageGallery + filters + ImageDetailModal	All components

7.3 Key Interfaces for Incremental Development

These interfaces define the contracts between modules. Implement stubs first, then fill in real logic. The system is runnable at every phase.

```
// IModelGateway – Gateway <-> ComfyUI interface
interface IModelGateway
{
  submitWorkflow(workflow: ComfyUIWorkflow): Promise<{ promptId: string }>;
  getJobStatus(promptId: string): Promise<JobStatus>;
  getOutputImage(promptId: string): Promise<Buffer>;
  getProgress(promptId:
```

```
string): AsyncIterable<ProgressEvent>; getLoadedModel(): Promise<string |
null>; isHealthy(): Promise<boolean>; } // IOrchestrator – Ion reasoning
pipeline interface IOrchestrator { processRequest(userInput: UserInput):
Promise<GenerationRequest>; getReasoningChain(requestId: string):
Promise<ReasoningStep[]>; } // IJobQueue – Async job management interface
IJobQueue { enqueue(request: GenerationRequest): Promise<{ jobId: string;
position: number }>; getStatus(jobId: string): Promise<JobState>;
cancel(jobId: string): Promise<void>; onComplete(jobId: string):
Promise<GenerationResponse>; getQueueDepth(): Promise<number>; } //
IImageStore – Storage abstraction interface IImageStore { save(imageBuffer:
Buffer, metadata: ImageMetadata): Promise<StorageRecord>; load(imageId:
string): Promise<Buffer>; getThumbnail(imageId: string): Promise<Buffer>;
list(filters: GalleryFilters): Promise<StorageRecord[]>; delete(imageId:
string): Promise<void>; }
```

MockModelGateway implementation: Returns a pre-generated 1024×1536 test PNG, fixed seed: 42, simulated 8500ms inference time (actually waits 100ms), predictable promptId. Activated by COMFYUI MOCK=true.

7.4 Environment & Config Strategy

```
# — ComfyUI Connection — COMFYUI_HOST=http://localhost:8188
COMFYUI_WS=ws://localhost:8188/ws COMFYUI MOCK=false # — Default Model
Settings — DEFAULT_CHECKPOINT=noobai-xl-vpred-v1.0
DEFAULT_PREDICTION_TYPE=v_prediction DEFAULT_VAE=sdxl-vae-fp16-fix
DEFAULT_SAMPLER=euler DEFAULT_SCHEDULER=normal DEFAULT_STEPS=28
DEFAULT_CFG=5.0 DEFAULT_CFG_RESCALE=0.2 DEFAULT_WIDTH=1024
DEFAULT_HEIGHT=1536 DEFAULT_CLIP_SKIP=2 # — Queue —
REDIS_URL=redis://localhost:6379 MAX_QUEUE_SIZE=100 MAX_CONCURRENT_JOBS=2
JOB_TIMEOUT_MS=120000 UPSCALE_JOB_TIMEOUT_MS=300000 # — Storage —
IMAGE_STORAGE_PATH=./storage/images THUMBNAIL_STORAGE_PATH=./storage/thumbs
METADATA_DB_URL=sqlite:./storage/metadata.db # — Safety —
SAFETY_ENABLED=true SAFETY_NSFW_THRESHOLD=0.7 RATE_LIMIT_PER_HOUR=30 # —
Logging — LOG_LEVEL=info LOG_FORMAT=json
OTEL_ENDPOINT=http://localhost:4317
```

Adding a New Model — Zero Code Changes

Drop the .safetensors file into /comfyui/models/checkpoints/. Add an entry to models.config.ts specifying: file name, display name, base model family, prediction type, recommended sampler, recommended CFG, quality tag convention, and recommended resolution. No other code changes needed — the workflow builder and prompt assembler read from this config automatically.

8. Anime-Specific Design

8.1 Philosophy

- The system is **anime-first**: every default, every optimization, every test case assumes anime output
- Non-anime styles (semi-realistic, painterly) are first-class citizens but not the default path
- The tag system follows **Danbooru conventions**—the lingua franca of anime image generation
- **Character accuracy** is a primary quality metric, not an afterthought
- The Illustrious XL model family was chosen specifically because it was trained on Danbooru2023—the largest curated anime tag dataset

8.2 Style Primitive Library

Seven production-ready style primitives. Each is a complete, tested configuration that reliably produces high-quality output in its aesthetic domain.

#	Style Family	Description	Ideal Use Case
1	Cinematic Niji-Grade Anime	High-detail, cinematic composition with dramatic lighting, vivid saturated colors, and clean linework. Equivalent to high-budget anime film quality.	Hero images, key visuals, dramatic scenes
2	Soft Pastel Shoujo	Delicate, dreamy aesthetic with soft pastel colors, gentle lighting, floral motifs, and ethereal atmosphere.	Romantic scenes, emotional portraits, gentle moods

#	Style Family	Description	Ideal Use Case
3	Gritty Seinen	Dark, mature aesthetic with high contrast, muted palette, and dramatic shadows.	Action, thriller, mature narrative content
4	Retro 90s Cel-Anime	Nostalgic 1990s aesthetic with cel-shading, warm VHS-like tones, and rougher linework evoking classic OVAs.	Nostalgia, retro aesthetics, classic anime homage
5	Semi-Realistic 2.5D	Hybrid style blending anime character design (large eyes, stylized hair) with realistic lighting and skin textures.	Character portraits, mixed-media, game art
6	Painterly / Watercolor	Traditional media aesthetic with visible brushstrokes, soft edges, and artistic imperfection mimicking hand-painted illustration.	Artistic scenes, book illustration, concept art
7	LoFi Aesthetic	Cozy, atmospheric scenes with warm lighting, everyday settings, and a relaxed mood.	Slice of life, cozy scenes, ambient backgrounds

8.3 Prompt Patterns for Reliable Anime Output

These patterns are battle-tested across thousands of generations on the Illustrious XL ecosystem.

27. **Quality tags first, always.** Lead every prompt with quality tags before content tags. The model was trained with quality tags at the front.

28. **Danbooru format.** Lowercase, underscores within multi-word tags, comma-separated: `1girl, long_hair, blue_eyes, school_uniform`
29. **Tag ordering matters.** Follow this hierarchy: quality → character count → character name → hair → eyes → expression → pose → clothing → accessories → background → lighting → style modifiers
30. **Negative prompt is non-negotiable.** Always include the base negative prompt (anatomy fixes + quality floor). Omitting it produces significantly worse results.
31. **Be explicit about characters.** Specify hair color, hair length, eye color, expression, and pose explicitly rather than relying on the model to infer.
32. **Be explicit about scenes.** Specify time of day, weather, camera angle, and background detail level.
33. **Use weight syntax sparingly.** (`tag:1.3`) for key emphasis only. Overweighting causes artifacts. Stay in the 0.5–1.4 range.
34. **Avoid contradictions.** Don't mix `realistic` + `chibi`, or `dark atmosphere` + `pastel colors`. You should detect and warn about contradictory tags.

8.4 LoRA Strategy

LoRA Type	Purpose	Weight Range	Example
Character LoRA	Consistent reproduction of specific characters	0.6–0.8	<code>character-hatsune-miku:0.75</code>
Style LoRA	Modify overall aesthetic (linework, coloring, rendering)	0.5–0.8	<code>style-thick-linework:0.6</code>
Concept LoRA	Add specific visual concepts not in base model	0.5–0.7	<code>concept-micro-details:0.5</code>

Stacking rules:

- Maximum 3 LoRAs per generation to avoid quality degradation
- Total combined weight should not exceed 2.0 (e.g., $0.8 + 0.7 + 0.5 = 2.0$)
- All LoRAs must be trained on or compatible with the Illustrious XL base architecture
- Ion should track available LoRAs and suggest relevant ones based on user intent and detected characters

9. Supporting Multiple Style Families Without Backend Fragmentation

9.1 Single Engine, Many Styles

This is a critical architectural principle: **all 7 styles run through the exact same ComfyUI pipeline**. There is no branching infrastructure. No separate endpoints. No conditional model loading based on style.

- Style differentiation happens **entirely in the prompt layer** — positive tags, negative tags, and generation parameters
- The ComfyUI workflow structure is identical for all styles — only the node values change
- Checkpoint swapping is **optional** and occurs at the workflow level, not the infrastructure level
- A style preset is just a data object. It does not require code. It does not require deployment.

9.2 Style Preset Architecture

Presets are stored as typed JSON objects in `/shared/style-presets.ts`:

```
interface StylePreset {  id: string;                                //
"cinematic_niji"  name: string;                                // "Cinematic Niji-
```

```
Grade Anime"    description: string;                // Human-readable
description    example_thumbnail: string;           // Path to example
output image   positive_prefix: string;             // Tags prepended to
user prompt    negative_additions: string;          // Tags added to base
negative       sampler: "euler" | "ddim";           // Recommended sampler
steps: number; // Recommended step count             cfg_scale:
number;        // Recommended CFG scale              default_resolution: {
// Default dimensions    width: number;    height: number;    };
checkpoint_override?: string; // Optional: force specific checkpoint
lora_stack?: LoRAConfig[];    // Optional: auto-apply LoRAs }
```

Merge order: lon assembles the final prompt as: `quality_tags` (from checkpoint config) + `positive_prefix` (from style preset) + `user_tags` (from user input) = final positive prompt.

9.3 Checkpoint-Aware Style Adaptation

When a user selects a checkpoint that uses different prompt conventions, lon automatically adapts the style preset’s quality tags. This is handled by a simple lookup table in `prompt-assembler.ts`:

```
const QUALITY_TAG_MAP: Record<string, string[]> = {  "noobai":
["masterpiece", "best quality", "absurdres"],  "animaginexl":
["masterpiece", "high score", "great score", "absurdres"],  "pony":
["score_9", "score_8_up", "score_7_up"],  "illustrious": ["masterpiece",
"best quality", "absurdres"], }; const SOURCE_TAG_MAP: Record<string,
string> = {  "pony": "source_anime", // Required for Pony V6 };
```

Adding a new style: Add a new `StylePreset` object to the array in `style-presets.ts`. No code changes. No deployment. The style appears in the UI automatically.

10. Migration Plan: Current Stack → New Architecture

10.1 Phase 0: Environment Setup (Day 1)

Task	Details	Time
Install ComfyUI	Clone repo into <code>/comfyui</code> , install Python dependencies, verify launch	30 min

Task	Details	Time
Download NoobAI-XL v-pred 1.0	6.62 GB .safetensors → /comfyui/models/checkpoints/	15 min
Download VAE	sdxl-vae-fp16-fix.safetensors → /comfyui/models/vae/	5 min
Download upscaler	4x-UltraSharp.pth → /comfyui/models/upscale_models/	5 min
Verify generation	Open ComfyUI web UI, load NoobAI-XL, generate test image with standard prompt, confirm quality	15 min
Install Redis	Local Redis instance for job queue development	10 min
Initialize project	Create /src/image-gen/ directory structure, init TypeScript config, install dependencies (zod, bullmq, sharp, ws)	30 min

Day 1 Exit Criteria

ComfyUI generates a high-quality anime image via its native web UI using NoobAI-XL v-pred with masterpiece, best quality, absurdres, 1girl at 1024×1536, Euler sampler, 28 steps, CFG 5.0. Image quality visually exceeds current Flux Schnell output. If this fails, debug before proceeding.

10.2 Phase 1: Parallel Run (Week 1-2)

- Build /shared/* contracts and /config/* files (Phase 1 of build order)
- Build ComfyUI client and workflow builder (Phase 2 of build order)
- Create MockModelGateway for CPU-only testing
- Build minimal end-to-end: hardcoded prompt → ComfyUI client → image saved to disk

- Keep old Flux Schnell + SD pipeline running as fallback—do not disable yet
- **Validation gate:** Generate 50 test images across all 7 styles on the new pipeline. Compare quality with old pipeline outputs. New pipeline must match or exceed on at least 80% of comparisons.

10.3 Phase 2: Ion Integration (Week 3-4)

- Build Ion orchestration modules (Phase 3 of build order)
- Wire Ion's reasoning into the orchestration layer
- Build prompt templates and tag expansion for all 7 style families
- Build the job queue (Phase 4 of build order)
- Test Ion's prompt adaptation across all 7 styles + all 4 checkpoint variants
- **Validation gate:** Natural language prompts like “draw a warrior in a sunset” produce high-quality anime output with correct tag expansion and style application.

10.4 Phase 3: Full Cutover (Week 5-6)

- **Disable old Flux Schnell and SD pipelines**—all generation routes through new architecture
- Build post-processing pipeline (Phase 5 of build order)
- Build frontend components (Phase 6 of build order)
- Migrate gallery/history data from old storage format to new metadata database
- Deploy to production environment

10.5 Phase 4: Polish (Week 7-8)

- Add structured logging and OpenTelemetry tracing
- Performance tuning: optimize ComfyUI workflow graph, tune queue concurrency

- LoRA library curation: download and test top-rated LoRAs for each style family
- Build Danbooru tag autocomplete database for prompt editor
- Write developer documentation for adding new models, styles, and LoRAs
- End-to-end integration tests covering all 7 styles, all error codes, and edge cases

10.6 Migration Risk Mitigation

Risk	Impact	Mitigation
NoobAI-XL quality doesn't meet expectations	High	Alternative checkpoints (AniagineXL 4.0, WAI-Illustrious) are drop-in replacements. Test multiple during Phase 1.
VRAM insufficient for SDXL on target GPU	High	fp16 inference requires 8 GB minimum. Enable <code>--lowvram</code> flag in ComfyUI for aggressive offloading. Budget for GPU upgrade if needed.
ComfyUI API instability	Medium	Pin ComfyUI version. Health check with auto-restart. MockModelGateway for development independence.
V-prediction configuration errors	Medium	Workflow builder hardcodes v-pred settings for NoobAI family. Extensive testing in Phase 1.
LoRA compatibility issues	Low	LoRA manager validates base model compatibility before injection. Graceful fallback: skip incompatible LoRA, log warning.

Ionirix Image Generation Architecture v1 — /docs/image-gen-architecture-v1.md

Author: Mirnes | Date: May 8, 2026 | Status: Implementation-Ready

© 2026 Ionirix. Internal architecture document.